TOPICS IN CYBERSECURITY

JAVASCRIPT

CONTENTS

- Language introduction
- Brief history from Vanilla Javascript to ES7.
- Variable declarations/initialization.
- Collections and their operations.
- Functions.
- Classes (OOP)
- Callbacks, promises, async/await.
- DOM/BOM
  - Event listeners
  - Selecting elements
  - Updating DOM

INTRODUCTION TO JAVASCRIPT

- JavaScript was invented by Brendan Eich in 1995, and became an ECMA standard in 1997.
- ECMAScript is the official name of the language.
- ECMAScript versions have been abbreviated to ES1, ES2, ES3, ES5, and ES6.
- Since 2016, versions are named by year (ECMAScript 2016, 2017, 2018, 2019, ..2024).

- *For more information:*
- *https://en.wikipedia.org/wiki/ECMAScript_version_history*
- *Book: Web Programming with HTML5, CSS and JavaScript by John Dean. (Chapter 8)*

VARIABLES

- Variables in javascript can be declared using following three keywords:
  - var (global scope)
  - let (block scope)
  - const (const is an augmentation of let in that it prevents re-assignment to a variable.

VARIABLES

- The variable's type is determined dynamically by the type of the value that's assigned into the variable. For example:
- `name = "Mia Hamm"; careerGoals = 158;`
- JavaScript is known as a *loosely typed language*, or a *dynamically typed language*
- You can assign different types of values into a variable at different times during the execution of a program.
- For example, it would be legal to assign a string to `name` and then later assign a number to `name`.
- Proper coding conventions dictate that you don't do that, because it can lead to code that's difficult to understand.

IDENTIFIER

- An identifier is the technical term for a program component's name—the name of a function, the name of a variable etc.
- Identifiers must consist entirely of letters, digits, dollar signs ($), and/or underscore (_) characters.
- The first character must not be a digit.
- Identifier names are case sensitive.
- Reserved words are not allowed.

DIFFERENCE BETWEEN VAR, LET AND CONST

	Scope	Redeclare	Reassign	Hoisted	Binds this
var	No	Yes	Yes	Yes	Yes
let	Yes	No	Yes	No	No
const	Yes	No	No	No	No

DATA TYPES IN JAVASCRIPT

- String
  Number
  Bigint
  Boolean
  Undefined
  Null
  Object

```javascript
// Numbers:
let length = 16;
let weight = 7.5;

// Strings:
let color = "Yellow";
let lastName = "Johnson";

// Booleans
let x = true;
let y = false;

// Object:
const person = {firstName:"John", lastName:"Doe"};

// Array object:
const cars = ["Saab", "Volvo", "BMW"];

// Date object:
const date = new Date("2022-03-25");
```

OBJECT DESTRUCTURING

- Basic Object Destructuring

```
const person2 = { name: "Ali", age: 22, city: "Lahore" };

const { name, age } = person2;
console.log(name); // "Ali"
console.log(age);  // 22
```

OBJECT DESTRUCTURING

- Rename variables while destructuring

```
const user = { id: 101, username: "najla" };

const { id: userId, username: userName } = user;
console.log(userId);   // 101
console.log(userName); // "najla"
```

OBJECT DESTRUCTING

- Default values

```
const student1 = { sname: "Haider", grade: "B" };
const student2 = { sname: "Haider"};
const { sname, grade = "A" } = student1;
const {sname1, grade1 = "A"} = student2;
console.log(sname);  // "Haider"
console.log(grade); // "B", not "A" since grade exists in
student

console.log(sname1);  // "Haider"
console.log(grade1); // "A"
```

OBJECT DESTRUCTURING

- Nested object desctructuring

```
const employee = {
  id: 1,
  info: { department: "IT", position: "Developer" }
};

const { info: { department, position } } = employee;
console.log(department); // "IT"
console.log(position);   // "Developer"
```

OBJECT DESTRUCTURING

- Destructuring in Function Parameters

```
function printUser({ name, age }) {
  console.log(`${name} is ${age} years old.`);
}

const person2 = { name: "Ali", age: 22, city: "Lahore" };
printUser(person2); // "Ali is 22 years old."
printUser({name:"Meesam", age: 4}); // "Ali is 22 years
old."
```

OPERATORS IN JAVASCRIPT

- Arithmatic Operators
- Used for mathematical calculations

+ // addition

- // subtraction

* // multiplication

/ // division

% // modulus (remainder)

** // exponentiation

++ // increment

-- // decrement

COMPARISON OPERATORS

- Used to compare two values – return true or false.

== // equal to (performs type coercion)

=== // strict equal to (no type coercion)

!= // not equal to

!== // strict not equal to

> // greater than

< // less than

>= // greater than or equal to

<= // less than or equal to

```
console.log("5 == '5'",5 == '5');  // true
console.log("5 === '5'", 5 === '5'); // false

console.log("5 != '5'", 5 != '5'); // false
console.log("5 !== '5'", 5 !== '5'); // true
```

LOGICAL OPERATORS

- Used for boolean logic

&& // AND

|| // OR

! // NOT

ASSIGNMENT OPERATORS

- Used to assign or update values

= // assign

+= // add and assign

-= // subtract and assign

*= // multiply and assign

/= // divide and assign

%= // modulus and assign

**= // exponent and assign

BITWISE OPERATORS

- Work on binary (bit-level) representations.

& // AND

| // OR

^ // XOR

~ // NOT

<< // left shift

>> // right shift

>>> // unsigned right shift

STRING OPERATORS

- Mainly + for concatenation

```
"Hello" + " " + "World"; // "Hello World"
```

- Use `` for using value of variables inside strings

```
let name = “Ali”;
console.log(`Person’s name is: ${name}`);
```

TERNARY OPERATOR

- A compact form of if-else

condition ? valueIfTrue : valueIfFalse;

TYPE OPERATORS

- Used for type checking or conversion

typeof // returns the type of a variable

instanceof // checks if an object is an instance of a class

```
class Student {}
class Teacher {}

const s = new Student();

console.log(typeof s);  // "object"

console.log(s instanceof Student); // true
console.log(s instanceof Teacher); // false
```

SPREAD AND REST OPERATOR (…)

- The … (spread or rest) operator in JavaScript is very powerful — it behaves differently depending on context.
- 1. Spread Operator (expands elements)
  - Used to expand arrays or objects.

```
//EXAMPLE WITH ARRAYS
const numbers = [1, 2, 3];
const moreNumbers = [...numbers, 4, 5];

console.log(moreNumbers); // [1, 2, 3, 4, 5]
```

```
//EXAMPLE WITH OBJECTS
const person = { name: "Ali", age: 20 };
const updatedPerson = { ...person, city: "Lahore" };

console.log(updatedPerson);
// { name: "Ali", age: 20, city: "Lahore" }
```

SPREAD AND REST OPERATOR (…)

- 2. Rest Operator (collects elements)
  - Used to gather remaining elements into an array or object.

```
function sum(...numbers) {
    let total = 0;
    for(let i = 0; i < numbers.length; i++) {
        total += numbers[i];
    }
    return total;
}

console.log(sum(1, 2, 3, 4)); // 10
```

```
//EXAMPLE IN DESTRUCTURING
const [first, ...rest] = [10, 20, 30, 40];
console.log(first); // 10
console.log(rest);  // [20, 30, 40]
```

NULLISH COALESCING AND OPTIONAL CHAINING

- Modern safety operators

?? // returns right side if left side is null or undefined

?. // safely access nested properties without causing errors

```
let name = null;
let displayName = name ?? "Guest";

console.log(displayName); // "Guest"

const user = {
  name: "Ali",
  address: {
    city: "Lahore"
  }
};

console.log(user.address.city);      // "Lahore"
console.log(user.contact.email);  // Error: Cannot read property 'email' of undefined
console.log(user.contact?.email);    // undefined (no error!)
```

FUNCTIONS

- Function Declaration
  - A named function that can be called anywhere

```
function greet() {
  console.log("Hello!");
}

greet(); // Output: Hello!
```

- Function Expression
  - Stored in a variable – not hoisted

```
const greet = function() {
  console.log("Hi there!");
};

greet(); // Output: Hi there!
```

FUNCTIONS

- Arrow Functions (ES6)
  - Shorter text. Often used in modern javascript.

```
const add = (a, b) => a + b;
console.log(add(2, 3)); // Output: 5
```

- Callback Function
  - Function passed as an argument to another function.

```
function greet(name) {
  console.log("Hello, " + name);
}

function processUser(callback) {
  const user = "Ali";
  callback(user);
}

processUser(greet); // Output: Hello, Ali
```

FUNCTIONS

- Anonymous functions
  - Function without a name, used in callbacks

```
setTimeout(function() {
  console.log("This runs after 2 seconds");
}, 2000);
```

- Immediately Invoked Function Expression (IIFE)
  - Runs as soon as its defined.

```
(function() {
  console.log("I run immediately!");
})();
```

ARRAYS

- `const cars = ["Saab", "Volvo", "BMW"];`
- Array indexes start with 0.
- You can also create an array, and then provide the elements:
-
```
const cars = [];
cars[0]= "Saab";
cars[1]= "Volvo";
cars[2]= "BMW";
```
- `const cars = new Array("Saab", "Volvo", "BMW");`
- Arrays are objects (type of array is object).
- You can have different data types in an array: `const person = ["John", "Doe", 46];`

ARRAY METHODS

- push() //pushes an element at the end
- pop() //returns last element
- length() //size of array
- at() //returns elements at an index mentioned in brackets
- forEach() //performs an operation on each element of an array
- map() //executes the same code on every element in an array and returns a new array with the updated elements.
- .filter() checks every element in an array to see if it meets a certain criteria and returns a new array with the elements that return truthy for the criteria.

ARRAY METHODS

```
// Initial array
let numbers = [10, 20, 30];

// 1 push() → adds element at the end
numbers.push(40);
console.log("After push:", numbers); // [10, 20, 30, 40]

// 2 pop() → removes and returns the last element
let last = numbers.pop();
console.log("Popped element:", last); // 40
console.log("After pop:", numbers);   // [10, 20, 30]

// 3 length → gives array size
console.log("Length:", numbers.length); // 3

// 4 at() → gets element by index (supports negatives too)
console.log("Element at index 1:", numbers.at(1));   // 20
//-ve index counts from end, -1 for last element
console.log("Last element using at():", numbers.at(-1)); // 30
```

ARRAY METHODS

```
// 5 forEach() → run a function on each element (no return)
numbers.forEach(num => console.log("forEach:", num * 2));
// Output: 20, 40, 60

// 6 map() → returns new array with transformed values
let doubled = numbers.map(num => num * 2);
console.log("Mapped array:", doubled); // [20, 40, 60]

// 7 filter() → returns new array with elements meeting a condition
let filtered = numbers.filter(num => num > 15);
console.log("Filtered array:", filtered); // [20, 30]
```

ARRAY METHODS

```
// reduce() takes a callback (accumulator, currentValue) and an initial value (0).
// It adds all elements to get the total sum.
let marks = [85, 33, 67, 90, 45];
//if the arrow function is written without curly brackets, the value is returned
automatically, otherwise we need to use return statement.
let sum = marks.reduce((total,mark)=>{return total + mark}, 0);
sum = marks.reduce((total,mark)=> total + mark, 0);//without brackets, auto return
console.log("Average marks:", sum/marks.length);
```

ITERATORS

- For of – used for strings and arrays
- For in – used for objects

```
//for of is used for strings and arrays
for (const fruit of fruits) {
  console.log(fruit);
}
for (const fruit of "mango") {
  console.log(fruit);
}

//for in is used for objects
const car = { make: "Toyota", model: "Camry", year: 2020 };

console.log(car["make"]);

for (const key in car) {
  console.log(`${key}: ${car[key]}`);
}
```

CLOSURES

A closure in JavaScript is created when a function remembers and accesses variables from its outer scope, even after that outer function has finished executing.

```javascript
//CLOSURES
let count = 0;
// count is a global variable, meaning any part of the program can accidentally modify or reset it.
// This breaks data encapsulation and makes debugging harder in larger applications.
function increaseCount() {
  count++;
  console.log("Count:", count);
}

increaseCount(); // Count: 1
increaseCount(); // Count: 2
increaseCount(); // Count: 3

function outerFunction (){
  let count = 0; // count is now private to outerFunction
  function innerFunction(){
    count++;
    console.log("Count:", count);
  }
  return innerFunction;
}
const increment = outerFunction();
increment();

// Here, count is private to outerFunction and can only be modified by innerFunction.
// This encapsulation prevents accidental interference from other parts of the program.
```

OUTPUT

- JavaScript can "display" data in different ways:
- Writing into an HTML element, using innerHTML.

```
document.getElementById("demo").innerHTML = 5 + 6;
```

- Writing into the HTML output using document.write(). It deletes all existing HTML.

```
<button type="button" onclick="document.write(5 + 6)">Try it</button>
```

- Writing into an alert box, using window.alert().

```
window.alert(5 + 6);
```

- Writing into the browser console, using console.log().

```
console.log(5 + 6);
```

THE <SCRIPT> TAG

- In Html, Javascript code is inserted between <script></script> tags.
- You can place these tags either in both head and body sections.
-
```
<script>
     document.getElementById("demo").innerHTML = "My First
     JavaScript";
</script>
```